HCLTech Internship Project Report

Cyber Security Engineering Division

HCLTech

CogniSOC: AI-Powered SOC Threat Detection and Response System

---

A Project Report

Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Technology in Cyber Security

Submitted By:

Ankit Singh
Roll No.:
B.Tech CSE (Cyber Security)
Batch: 2023–2027

Under the Guidance of:

Mr. Harinder Sodhi
Trainer & Industry Guide
HCLTech

Academic Year 2024–2025

CERTIFICATE

This is to certify that the project titled **"CogniSOC: AI-Powered SOC Threat Detection and Response System"** has been carried out by **Ankit Singh** (Roll No.:) under my supervision and guidance during the academic year 2024–2025 as part of the Internship Program at **HCLTech**.

This project is a bona fide record of the work done by the candidate and has not been submitted elsewhere for any other degree or diploma.

Mr. Harinder Sodhi	**Project Manager**
Trainer & Industry Guide	Cyber Security Engineering Division
HCLTech	HCLTech

Date: Place: Greater Noida

ACKNOWLEDGMENT

I would like to express my sincere gratitude to all those who contributed to the successful completion of this project.

First and foremost, I am deeply thankful to my trainer and industry guide, **Mr. Harinder Sodhi** at **HCLTech**, for providing invaluable guidance, constant encouragement, and constructive feedback throughout the course of this project. His expertise in cybersecurity and willingness to dedicate time for discussions were instrumental in shaping this work.

I extend my heartfelt thanks to the **Head of the Department of Cyber Security** for providing the necessary infrastructure and laboratory resources, including access to the SOC lab environment with VMware virtualization, Splunk Enterprise, and TheHive SOAR platform.

I am grateful to **Dr. A.P.J. Abdul Kalam Technical University, Lucknow** for the academic framework and curriculum that inspired the application of machine learning in the domain of cybersecurity operations.

I also wish to acknowledge the open-source communities behind **Splunk**, **Sysmon**, **Suricata**, **TheHive**, **Sigma Rules**, **MITRE ATT&CK**, **Atomic Red Team**, and **scikit-learn**, whose freely available tools and datasets made this research possible at zero cost.

Finally, I thank my family and friends for their unwavering support and patience during the intensive development phase of this project.

Ankit Singh
Cyber Security Intern
HCLTech

ABSTRACT

Modern Security Operations Centers (SOCs) are overwhelmed by an exponentially growing volume of security alerts generated by endpoint detection tools such as **Sysmon** and network intrusion detection systems such as **Suricata**. Industry reports indicate that the average enterprise SOC receives over 11,000 alerts per day, of which more than 40% are false positives. This alert fatigue leads to critical threats being missed, delayed, or deprioritized by human analysts.

This project presents **CogniSOC**, an AI-powered SOC threat detection and response system that augments traditional rule-based SIEM detection with **unsupervised machine learning-based behavioral anomaly scoring**. The system is built as a Python-based analytics engine that integrates with an existing Splunk Enterprise SIEM deployment. It fetches Sysmon and Suricata telemetry via the Splunk REST API, normalizes and parses heterogeneous log formats (including raw Sysmon XML and Suricata EVE JSON), and aggregates them into per-entity time-windowed behavioral feature vectors.

An **Isolation Forest** algorithm, trained on known-normal lab telemetry, assigns anomaly scores to each entity-time-window observation. The scoring pipeline incorporates a **baseline deviation calibration** mechanism specifically designed for low-variance lab environments, ensuring that obvious out-of-baseline behaviors (such as PowerShell execution, rare processes, and unexpected parent-child process chains) are not suppressed by a narrow training distribution.

The system further implements a **six-rule correlation engine** that maps anomalous findings to specific attack scenarios (Reconnaissance, PowerShell abuse, Brute Force, Data Exfiltration, LOLBin execution, and Malicious Process Execution), an **incident classifier** that categorizes findings into six enterprise incident types (Malware, Phishing, DDoS, Insider Threat, Brute Force, and Data Breach), and a **priority scoring engine** that assigns P1–P4 urgency levels. Findings are automatically pushed to **TheHive** SOAR platform as full investigation cases with pre-built analyst tasks, and simultaneously re-ingested into Splunk via **HTTP Event Collector (HEC)** for a unified SOC dashboard view.

The system was validated using **Atomic Red Team** attack simulations across six MITRE ATT&CK techniques and detected all simulated attack scenarios with anomaly scores exceeding the 90th-percentile threshold. A real-time monitoring mode with a Python Dash dashboard and a Splunk Studio dashboard ("SOC Command Center") provides analysts with an investigation queue, severity distribution charts, MITRE ATT&CK context, risk factor breakdowns, and one-click Splunk pivot queries.

Keywords: SOC, SIEM, Machine Learning, Isolation Forest, Anomaly Detection, Sysmon, Suricata, Splunk, TheHive, MITRE ATT&CK, Incident Response, Behavioral Analytics.

TABLE OF CONTENTS

Contents

LIST OF FIGURES

LIST OF TABLES

1 Introduction

1.1 Background and Motivation

A Security Operations Center (SOC) serves as the centralized nerve center for an organization's cybersecurity monitoring, detection, and incident response capabilities. SOC analysts are responsible for monitoring streams of security telemetry generated by firewalls, intrusion detection systems (IDS), endpoint detection and response (EDR) agents, and Security Information and Event Management (SIEM) platforms. The fundamental challenge facing modern SOCs is **alert fatigue**: the sheer volume of alerts generated by these tools far exceeds the capacity of human analysts to investigate each one in a timely manner.

According to IBM's 2024 Cost of a Data Breach Report, the global average cost of a data breach reached USD 4.88 million, with organizations taking an average of 194 days to identify a breach [1]. Palo Alto Networks' Unit 42 Incident Response Report found that in 45% of cases, attackers exfiltrated data within a single day of initial compromise [2]. These statistics underscore a critical gap: detection speed is a decisive factor in breach impact, yet SOC teams are drowning in noise.

Traditional SOC architectures rely primarily on **signature-based** and **rule-based** detection. Splunk correlation searches, Sigma rules, and Suricata IDS signatures fire alerts when telemetry matches known-bad patterns. While effective for known threats, these approaches suffer from three fundamental limitations:

1. **High false-positive rates:** Generic rules generate thousands of alerts daily, most of which are benign. A 2023 study by Tines found that SOC analysts spend 32% of their time investigating alerts that turn out to be false positives [3].
2. **Inability to detect novel threats:** Signature-based systems cannot detect zero-day exploits, fileless malware, living-off-the-land (LOLBin) attacks, or sophisticated adversary techniques that do not match existing rule patterns.
3. **No behavioral context:** Individual log lines are evaluated in isolation. A single PowerShell execution is not inherently suspicious, but a burst of 50 PowerShell invocations, combined with rare process names and unusual parent-child process relationships within a 15-minute window, is a strong behavioral signal that rules alone cannot capture.

This project addresses these limitations by introducing an **unsupervised machine learning** layer that operates alongside the existing rule-based detection stack. Rather than replacing

Splunk and Sigma, the system augments them with behavioral anomaly scoring that captures entity-level deviations from a trained baseline of normal activity.

1.2 Problem Statement

Design and implement an AI-powered behavioral analytics system that:

1. Integrates with an operational Splunk Enterprise SIEM deployment to ingest Sysmon and Suricata telemetry via the REST API.
2. Normalizes heterogeneous log formats (Sysmon XML, Suricata EVE JSON, flat Splunk key-value fields) into a unified event schema.
3. Extracts explainable behavioral features from aggregated entity-time-window observations.
4. Trains an Isolation Forest anomaly detection model on known-normal lab telemetry and scores new telemetry without retraining.
5. Correlates anomalous findings into attack-specific incidents mapped to the MITRE ATT&CK framework.
6. Automatically creates investigation cases with pre-built analyst tasks in TheHive SOAR platform.
7. Re-ingests ML findings into Splunk via HEC for unified dashboard visibility.
8. Provides real-time monitoring with auto-refreshing dashboards.

1.3 Objectives

1. To reduce SOC analyst alert fatigue by surfacing only behaviorally anomalous activity above a configurable threshold.
2. To detect attack patterns that evade traditional rule-based detection by analyzing behavioral deviations across multiple telemetry dimensions simultaneously.
3. To provide explainable anomaly scores with human-readable reason strings, risk factors, and MITRE ATT&CK context so that analysts understand **why** an alert was raised.
4. To automate the incident response workflow by creating structured TheHive cases with prioritized tasks.
5. To validate the system against real attack simulations using the Atomic Red Team framework.

1.4 Scope and Limitations

In scope:

- Detection of host-based attacks via Sysmon (process creation, network connections, DNS queries, file creation, registry modification).
- Detection of network-based attacks via Suricata IDS alerts.
- Integration with Splunk Enterprise (SIEM), TheHive (SOAR), and a custom Python Dash dashboard.
- Validation using Atomic Red Team simulations covering six MITRE ATT&CK techniques.

Out of scope:

- Cloud workload telemetry (AWS CloudTrail, Azure Activity Logs).
- Full endpoint detection and response (EDR) capabilities such as host isolation or process killing.
- Supervised classification (the model is unsupervised by design, as labeled attack data is scarce in real SOC environments).

1.5 Organization of the Report

This report is organized into ten chapters. Section 2 presents a literature review of existing SOC architectures, machine learning approaches for anomaly detection, and the MITRE ATT&CK framework. Section 3 describes the system architecture, including the 4-machine lab setup, technology stack, and data flow. Section 4 details the implementation of each software module with code excerpts and configuration. Section 5 explains the feature engineering and machine learning model design. Section 6 covers the correlation engine, incident classifier, and priority scoring. Section 7 presents the dashboard and visualization layer. Section 8 documents the testing methodology and results using Atomic Red Team. Section 9 discusses challenges encountered during development and their solutions. Section 10 provides conclusions and future work.

2 Literature Review

2.1 Security Operations Centers: Architecture and Challenges

The modern SOC operates on a three-tier analyst model. Tier-1 analysts perform initial alert triage and escalation; Tier-2 analysts conduct deeper investigation and threat hunting; Tier-3 analysts handle advanced incident response and forensics [16]. The primary tool in a SOC is the SIEM platform, which aggregates logs from multiple sources, normalizes them into a searchable index, and applies correlation rules to generate alerts.

Splunk Enterprise is one of the most widely deployed SIEM platforms in enterprise environments, with over 15,000 customers globally [17]. Splunk ingests machine data from endpoints, network devices, cloud services, and applications, and provides a powerful Search Processing Language (SPL) for querying and correlating events. However, Splunk's detection capabilities are fundamentally rule-based: an analyst or vendor must write a correlation search or alert condition for every threat scenario. This creates a dependency on known threat intelligence and leaves gaps for novel or evasive attack techniques.

2.2 Anomaly Detection in Cybersecurity

Anomaly detection techniques for cybersecurity can be broadly categorized into three approaches:

Statistical methods use distributional models to identify outliers. Z-score analysis, moving averages, and exponential smoothing are commonly applied to network traffic volume, login frequency, and process execution counts [4]. While computationally efficient, statistical methods struggle with multi-dimensional data and complex correlations between features.

Supervised machine learning methods, including Random Forests, Gradient Boosting, and deep neural networks, require labeled training data with both normal and attack examples. The NSL-KDD dataset [8] and CICIDS2017 dataset [9] have been widely used for this purpose. However, supervised approaches face a fundamental limitation in real SOC deployments: labeled attack data is expensive to obtain, quickly becomes outdated, and may not represent the specific attack patterns relevant to a given organization.

Unsupervised machine learning methods, including Isolation Forest [5], One-Class SVM [6], and autoencoders [7], learn a model of normal behavior from unlabeled data and flag deviations as anomalies. This approach is particularly well-suited to SOC environments because:

- Only normal (baseline) data is required for training, which is abundant in any production environment.
- The model generalizes to detect previously unseen attack patterns, including zero-day exploits.
- No continuous relabeling of training data is needed as the threat landscape evolves.

2.3 Isolation Forest Algorithm

The Isolation Forest algorithm, introduced by Liu, Ting, and Zhou in 2008 [5], is an ensemble method specifically designed for anomaly detection. Unlike distance-based or density-based methods, Isolation Forest exploits the fact that anomalies are **few and different**: they are easier to isolate (separate from the rest of the data) using random partitioning.

The algorithm constructs an ensemble of binary decision trees (isolation trees), where each tree recursively partitions the data by randomly selecting a feature and a split value. Anomalous data points, being rare and distant from normal clusters, require fewer splits to isolate and thus have shorter average path lengths across the ensemble. The anomaly score is derived from the average path length normalized by the expected path length under a random binary search tree model:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

where $E(h(x))$ is the expected path length for observation x, and $c(n)$ is the average path length of unsuccessful searches in a Binary Search Tree with n observations.

Isolation Forest was selected for this project due to its:

- **Linear time complexity** $O(t \cdot n)$ where t is the number of trees and n is the sample size, making it suitable for real-time SOC scoring.
- **Robustness to irrelevant features**, as random feature selection naturally handles high-dimensional data.
- **No requirement for distance computation**, which avoids the curse of dimensionality affecting kNN and DBSCAN-based detectors.

2.4 MITRE ATT&CK Framework

The MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) framework is a globally accessible knowledge base of adversary behavior based on real-world observations [13]. It categorizes adversary actions into 14 tactics (e.g., Initial Access, Execution, Persistence, Lateral Movement, Exfiltration) and hundreds of specific techniques and sub-techniques.

In this project, MITRE ATT&CK serves two critical roles:

1. **Attack simulation design:** Atomic Red Team tests are organized by ATT&CK technique IDs, allowing systematic coverage of the attack surface.
2. **Finding enrichment:** Anomalous findings are annotated with the most likely ATT&CK tactics and techniques based on the specific behavioral features that triggered the anomaly.

2.5 Sigma Detection Rules

Sigma is an open standard for SIEM detection rules, often described as "SNORT for logs" [14]. Sigma rules are written in YAML format and describe detection logic independent of any specific SIEM platform. They can be converted to Splunk SPL, Elastic Query DSL, Microsoft Sentinel KQL, and other query languages using tools such as `sigmac` and `pySigma`.

This project includes 13 custom Sigma rules covering attack scenarios including suspicious PowerShell execution, credential dumping, brute force, data exfiltration, DDoS, insider threats, lateral movement, defense evasion, LOLBin execution, malware execution, phishing documents, port scanning, and encoded PowerShell commands.

2.6 TheHive SOAR Platform

TheHive is an open-source Security Orchestration, Automation, and Response (SOAR) platform designed for SOC teams [15]. It provides case management, alert management, observable enrichment (via Cortex analyzers), and structured analyst workflows. In this project, TheHive serves as the automated incident response backend: ML findings are pushed as full investigation cases with pre-defined analyst tasks via TheHive's REST API.

2.7 Related Work

Study	Approach	Data Source	Limitation
Chandola et al. [4]	Survey of anomaly detection	Multiple	No SOC-specific implementation
Liu et al. [5]	Isolation Forest algorithm	Synthetic	Not applied to security telemetry
Mirsky et al. [10]	Autoencoder (Kitsune)	Network packets	No host-level (Sysmon) telemetry
Ring et al. [11]	Survey of IDS datasets	Network flows	No integration with SIEM/SOAR
Moustafa et al. [12]	UNSW-NB15 dataset	Network	Supervised; requires labeled data
This project (CogniSOC)	**Isolation Forest + correlation + SOAR**	**Sysmon + Suricata via Splunk**	**Lab environment only**

Table 1: Comparison of CogniSOC with related work in anomaly-based threat detection.

As shown in Table 1, existing academic work typically addresses anomaly detection in isolation—either focusing on the ML algorithm or the dataset—without building a complete, deployable SOC pipeline that integrates detection, correlation, incident response, and dashboard visualization. CogniSOC fills this gap by delivering an end-to-end system validated against real attack simulations in a production-grade Splunk/TheHive environment.

3 System Architecture

3.1 Development Methodology

The project followed an **Agile-Iterative** development methodology, adapted for a single-developer internship setting. Development was organized into five iterative sprints, each lasting approximately 2–3 weeks:

1. **Sprint 1 — Lab Setup and Telemetry Pipeline:** Installation and configuration of the 4-machine lab (VMware, Splunk, Sysmon, Suricata, TheHive). Verification of data flow from endpoints to the SIEM.
2. **Sprint 2 — Data Ingestion and Parsing:** Development of the Splunk REST API client and the multi-format event parser (Sysmon XML, Suricata JSON, flat fields).
3. **Sprint 3 — ML Engine and Feature Engineering:** Design of the 18-dimensional feature vector, Isolation Forest training, and the dual-score anomaly scoring architecture.
4. **Sprint 4 — Correlation, Classification, and SOAR Integration:** Development of the six-rule correlation engine, incident classifier, priority scorer, and TheHive connector.
5. **Sprint 5 — Dashboards, Testing, and Validation:** Development of Python Dash and Splunk dashboards, Atomic Red Team testing, Sigma rule authoring, and documentation.

Version control was managed using **Git** with meaningful commit messages for each feature. The repository was hosted on **GitHub** for backup and collaboration with the industry guide.

3.2 Lab Environment: The 4-Machine Setup

The system was developed and tested in a virtualized lab environment consisting of four machines, each serving a distinct role in the SOC pipeline. Table 2 summarizes the configuration.

Figure 1: Virtual machine hypervisor setup for the isolated SOC lab environment.

Machine	**Operating System**	**IP Address**	**Role**
Main PC (Host)	Windows 11, 16 GB RAM	Physical host	Development, ML model training, dashboard, report writing
Ubuntu VM (SIEM)	Ubuntu Linux	172.16.140.130	Splunk Enterprise (ports 8089, 8000, 8088, 9997) + The-Hive (port 9000)
Victim VM	Windows 10	Internal VM network	Sysmon endpoint + Splunk Universal Forwarder + Atomic Red Team
Attacker VM	Kali Linux	Internal VM network	Attack simulation: nmap, Metasploit, curl

Table 2: Lab environment machine configuration.

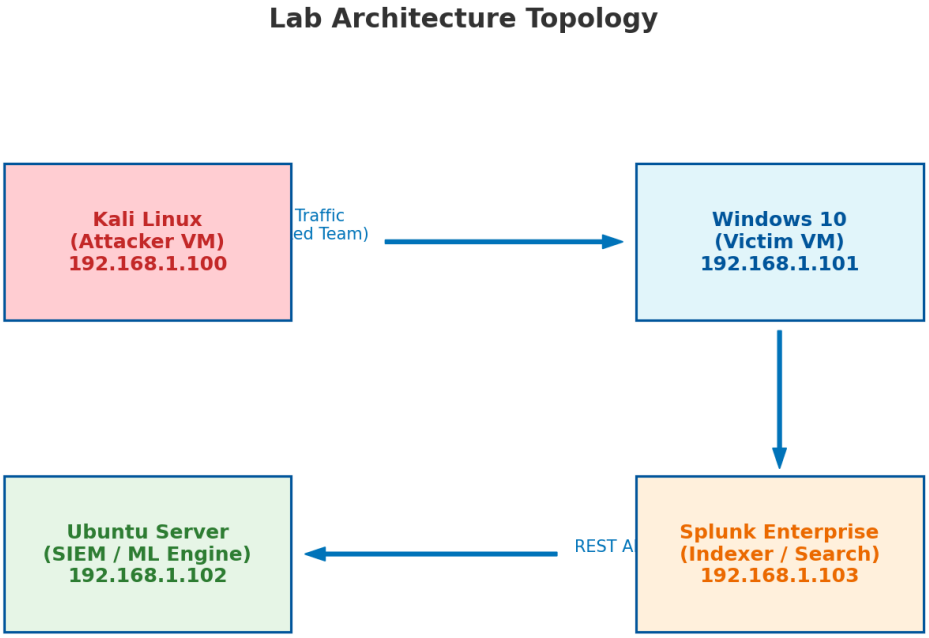

Figure 2: Lab network topology diagram.

3.3 Technology Stack

Component	Technology	Version
SIEM Platform	Splunk Enterprise	9.x (Free License)
Endpoint Telemetry	Microsoft Sysmon	15.x
Network IDS	Suricata	7.x
Log Forwarding	Splunk Universal Forwarder	9.x
SOAR Platform	TheHive	4.x
Programming Language	Python	3.10+
ML Framework	scikit-learn (Isolation Forest)	1.4.2
Data Processing	pandas, NumPy	2.2.2 / 1.26.4
Visualization	Plotly Dash, Seaborn, Matplotlib	2.x / 0.13.2 / 3.8.4
API Communication	requests, splunk-sdk	2.31.0 / 1.7.4
Attack Simulation	Atomic Red Team	Latest
Detection Rules	Sigma (13 custom rules)	YAML format
Version Control	Git + GitHub	Latest

Table 3: Technology stack used in CogniSOC.

3.4 System Data Flow

The complete data flow of the CogniSOC system follows a five-stage pipeline:

Stage 1 — Telemetry Generation: Attack simulations are executed on the Victim VM using Atomic Red Team. Sysmon captures process creation (Event ID 1), network connections (Event ID 3), DNS queries (Event ID 22), and file creation (Event ID 11) events. Suricata on the Ubuntu VM captures network-level alerts from traffic between the Attacker VM (Kali) and the Victim VM.

Stage 2 — Telemetry Ingestion: The Splunk Universal Forwarder on the Victim VM forwards Sysmon events to the Splunk Enterprise indexer on the Ubuntu VM via port 9997. Suricata's `eve.json` output is directly monitored by Splunk on the Ubuntu VM.

Stage 3 — ML Detection: The Python `soc_ml_engine` on the Main PC fetches recent telemetry from Splunk via the REST API (port 8089), parses and normalizes the events, extracts behavioral features, and scores them against the trained Isolation Forest baseline model.

Stage 4 — Correlation and Classification: Suspicious findings (anomaly score ≥ 90) are passed through the correlation engine, which maps behavioral patterns to specific attack scenarios. The incident classifier assigns enterprise categories, and the prioritizer assigns P1–P4 urgency levels.

Stage 5 — Response and Visualization: Correlated incidents are automatically pushed to TheHive as investigation cases with analyst tasks. Findings are also re-ingested into Splunk via HEC for the SOC Command Center dashboard.

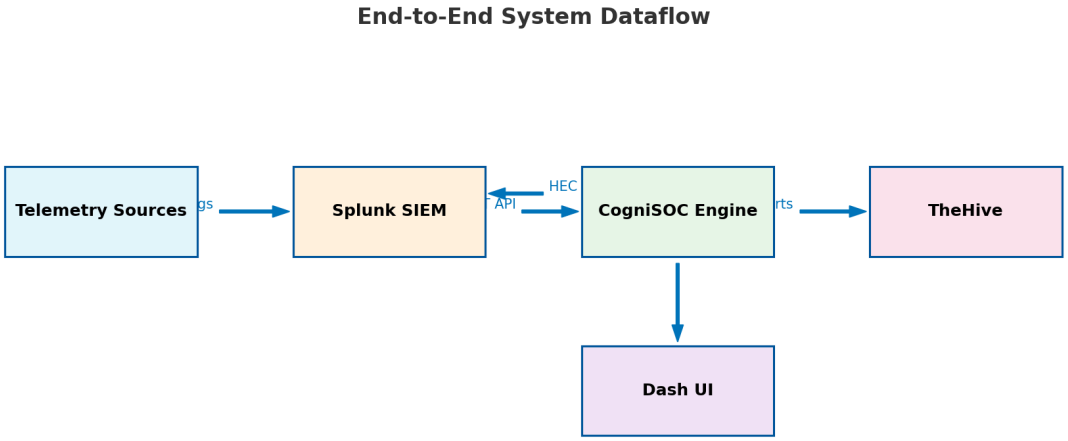

Figure 3: Complete system data flow from telemetry generation to incident response.

3.5 Software Module Architecture

The Python codebase is organized into six logical modules:

```
soc_ml_engine/
├── config/settings.py            – Configuration management
├── fetcher/splunk_fetcher.py     – Splunk REST API client
├── processing/features.py        – Event parsing & feature engineering
├── models/anomaly_model.py       – Isolation Forest training & scoring
├── correlation/
│   ├── correlator.py             – 6-rule attack correlation engine
│   ├── incident_classifier.py    – 6-category incident classifier
│   ├── prioritizer.py            – P1-P4 priority scoring
│   ├── timeline_builder.py       – Attack timeline reconstruction
│   └── report_generator.py       – Structured incident report
├── integration/
│   ├── thehive_connector.py      – TheHive SOAR case creation
│   └── splunk_hec_push.py        – Splunk HEC re-ingestion
```

```
├── dashboard.py                — Python Dash SOC dashboard
├── realtime.py                 — Real-time monitoring loop
└── main.py                     — CLI entry point
```

4 Implementation

This chapter presents the implementation details of each software module, including code excerpts, configuration, and screenshots demonstrating the working system.

SCREENSHOT: Splunk Enterprise login page or home screen on `http://172.16.140.130:8000` showing the Search & Reporting app.

File: `report/images/splunk_home.png`

Figure 4: Splunk Enterprise home screen on the Ubuntu SIEM VM.

4.1 Splunk Enterprise Configuration

4.1.1 Installation and Index Setup

Splunk Enterprise was installed on the Ubuntu VM and configured to receive forwarded logs on port 9997. The primary index `main` was configured to store both Sysmon and Suricata telemetry. The Splunk Free License was activated after the trial period expired, which disables user authentication but retains full search and indexing capabilities within the 500 MB/day ingestion limit.

SCREENSHOT: Splunk Web UI showing the main index with data summary — sourcetypes visible should include "XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" and "suricata".

File: `report/images/splunk_index_summary.png`

Figure 5: Splunk Enterprise data summary showing Sysmon and Suricata sourcetypes.

4.1.2 Splunk Universal Forwarder Configuration

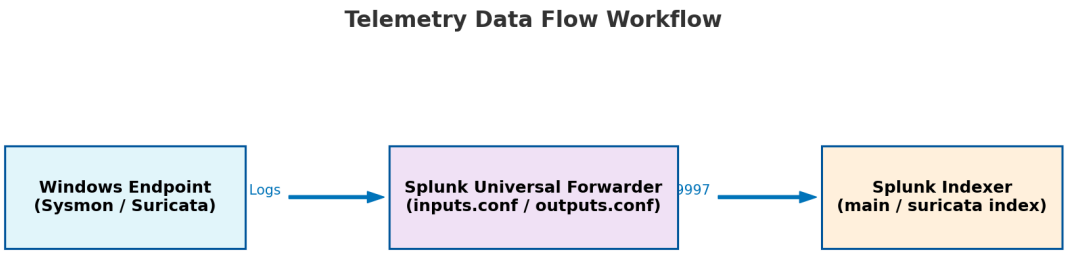

Figure 6: Telemetry collection and forwarding data flow.

The Splunk Universal Forwarder was installed on the Victim VM (Windows 10) and configured to forward Sysmon operational logs to the Ubuntu indexer. The critical configuration file is `inputs.conf`:

```
# C:\Program Files\SplunkUniversalForwarder\etc\apps\
#   Splunk_TA_microsoft_sysmon\local\inputs.conf
[WinEventLog://Microsoft-Windows-Sysmon/Operational]
index = main
disabled = 0
renderXml = true
```

The `renderXml = true` setting ensures that full Sysmon XML payloads are preserved in Splunk's `_raw` field, enabling the Python parser to extract all data fields (Image, CommandLine, ParentImage, etc.) without relying on Splunk's field extraction configuration.

SCREENSHOT: Victim VM terminal showing `splunk.exe list forward-server` output with 172.16.140.130:9997 listed as active.

File: `report/images/forwarder_status.png`

Figure 7: Splunk Universal Forwarder status showing active forwarding to the Ubuntu indexer.

4.1.3 Sysmon Installation and Configuration

Microsoft Sysmon (System Monitor) was installed on the Victim VM to provide detailed endpoint telemetry. Sysmon was configured with a community-standard configuration file (SwiftOnSecurity/sysmon-config) that captures:

- **Event ID 1** — Process Creation (with full command line and parent process)
- **Event ID 3** — Network Connection
- **Event ID 11** — File Creation
- **Event ID 13** — Registry Value Set
- **Event ID 22** — DNS Query

SCREENSHOT: Victim VM showing Sysmon service running in Services.msc
or `Get-Service Sysmon64` output showing Status: Running.

File: `report/images/sysmon_running.png`

Figure 8: Sysmon service running on the Victim VM.

4.1.4 Suricata Network Intrusion Detection

Suricata was installed and configured on the Ubuntu SIEM VM to monitor lab network traffic. It logs network events, including DNS, HTTP, and specific alert signatures, to `/var/log/suricata/eve.json`. The Splunk Universal Forwarder (or local Splunk instance) monitors this JSON file continuously. Suricata alerts provide critical context for the correlation engine to detect reconnaissance and data exfiltration.

SCREENSHOT: Ubuntu VM terminal showing `tail -f /var/log/suricata/eve.json`
output with NIDS alerts, or the Suricata service status.

File: `report/images/suricata_alerts.png`

Figure 9: Suricata NIDS capturing malicious traffic and outputting to `eve.json`.

4.2 Splunk REST API Client (`splunk_fetcher.py`)

The `SplunkFetcher` class provides a resilient REST API client for querying Splunk Enterprise. It creates asynchronous search jobs, polls for completion, and paginates through results. Key design decisions include:

- **Retry logic:** Configurable retry count with exponential backoff for transient network failures and Splunk 500-series errors.
- **SSL handling:** SSL verification is disabled by default (`verify=False`) because the lab Splunk instance uses a self-signed certificate.
- **Dual authentication:** Supports both bearer token and username/password authentication.
- **Broad search query:** The default search fetches both Sysmon and Suricata sourcetypes with all relevant fields, including the raw payload for XML/JSON parsing.

```
def _default_search(self, max_events):
    limit = f" | head {int(max_events)}" if max_events else ""
    return (
        f'search index="{self.config.index}" '
        '(sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" '
        'OR sourcetype="sysmon" OR sourcetype="suricata" '
        'OR source="*suricata*" OR event_type=*) '
        '| eval raw_payload=substr(_raw,1,5000) '
        '| fields _time host source sourcetype EventCode Image '
        'ParentImage CommandLine SourceIp DestinationIp '
        'DestinationPort src_ip dest_ip dest_port proto '
        'event_type signature alert* raw_payload'
        f"{limit}"
    )
```

SCREENSHOT: Terminal output showing the soc_ml_engine fetching telemetry from Splunk — the log lines showing "Created Splunk search job sid=…" and "Fetched N Splunk records".

File: `report/images/splunk_fetch_output.png`

Figure 10: Terminal output of the Splunk telemetry fetch operation.

4.3 Event Parsing and Normalization (`features.py`)

One of the most complex engineering challenges in this project was normalizing the diverse telemetry formats into a unified event schema. The system handles three distinct input formats:

1. **Sysmon XML:** Raw Windows Event Log XML embedded in Splunk's `_raw` field. Parsed using Python's `xml.etree.ElementTree` to extract `EventID`, `Image`, `CommandLine`, `ParentImage`, and other Sysmon-specific fields.
2. **Suricata EVE JSON:** Nested JSON objects from Suricata's `eve.json` log, flattened using recursive key expansion (e.g., `alert.signature` becomes both the dotted key and the base key `signature`).
3. **Flat Splunk key-value fields:** Pre-extracted fields from Splunk's field extraction pipeline, used as a fallback when raw parsing fails.

The `_normalize_record()` function implements a **merge-with-priority** strategy: raw-parsed fields are merged with Splunk-extracted fields, with the raw parse taking precedence for fields that exist in both sources. This ensures maximum data fidelity while maintaining resilience when raw payloads are truncated or unavailable.

```
def _normalize_record(record):
    raw_payload = record.get("_raw") or ""
    raw_fields = _parse_raw(raw_payload)  # XML or JSON
    merged = {**raw_fields, **{k: _first(v) for k, v in record.items()}}
    event_id = str(merged.get("EventCode") or merged.get("EventID") or "")
    image = str(merged.get("Image") or "")
    event_type = _event_type(merged, event_id)
    return {
        "timestamp": merged.get("_time"),
        "host": merged.get("host") or merged.get("Computer") or "",
        "event_type": event_type,
        "process_name": _basename(image),
        "command_line": merged.get("CommandLine") or "",
        # ... (12 additional fields)
    }
```

5 Feature Engineering and Machine Learning Model

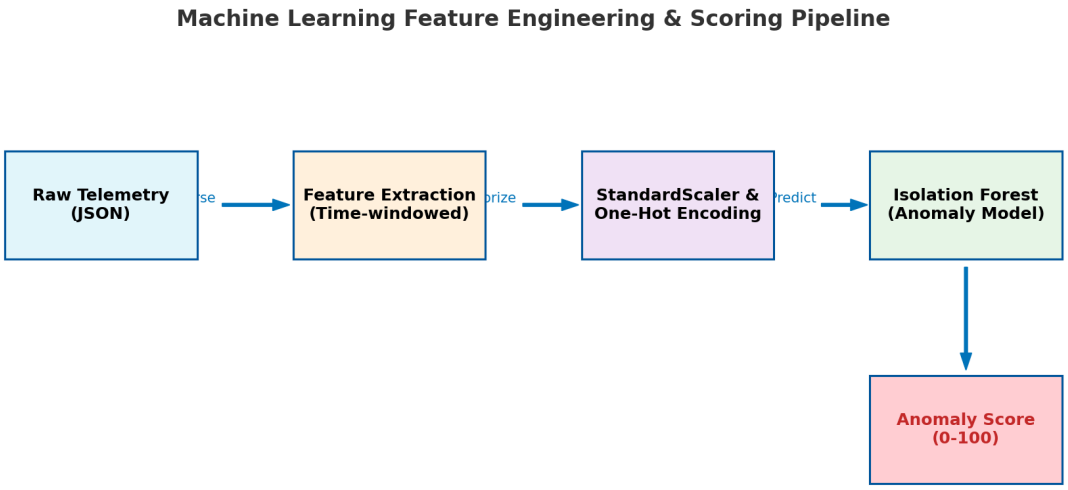

Figure 11: Machine Learning feature engineering and scoring pipeline.

5.1 Feature Engineering Philosophy

The CogniSOC system does not score individual log lines. Instead, it aggregates telemetry into **entity-time-window** observations, where an entity is either a hostname (for Sysmon events) or a source IP address (for Suricata events), and the time window is a configurable interval (default: 15 minutes). This design mirrors how a SOC analyst actually thinks: "this host behaved strangely during this interval."

5.2 Feature Vector: 18 Dimensions

Each entity-time-window observation is represented as an 18-dimensional numeric feature vector. Table 4 describes each feature.

Feature Name	Type	Description
event_count	Count	Total number of raw events in the window
process_event_count	Count	Sysmon Event ID 1 (Process Create) count
network_event_count	Count	Sysmon Event ID 3 (Network Connection) count
suricata_alert_count	Count	Suricata IDS alert count
powershell_count	Count	Processes matching powershell or pwsh
cmd_count	Count	Processes matching cmd.exe
unique_process_count	Count	Distinct process names observed
top_process_frequency	Count	Execution count of the most frequent process
unique_dest_ports	Count	Distinct destination ports (port diversity)
connection_count	Count	Total outbound network connections
unique_parent_child_pairs	Count	Distinct parent→child process relationships
rare_process_count	Count	Processes not seen in the baseline profile
unexpected_parent_child_count	Count	Parent→child pairs not seen in baseline
event_type_process_create	Ratio	Fraction of events that are process creates
event_type_network_connection	Ratio	Fraction of events that are network connections
event_type_suricata_alert	Ratio	Fraction of events that are Suricata alerts
event_type_dns_query	Ratio	Fraction of events that are DNS queries
event_type_file_created	Ratio	Fraction of events that are file creations

Table 4: Complete feature vector (18 dimensions) used by the Isolation Forest model.

All features are **numeric counts or ratios**, deliberately avoiding text-based or categorical features. This design choice ensures that every feature is directly explainable in a viva or SOC engineering review: "the anomaly score was high because powershell_count was 47 while the baseline mean was 0.2."

5.3 Baseline Profile

Before training the Isolation Forest, the system derives a **baseline profile** from the training data. This profile captures two sets of reference information:

1. **Common processes:** The set of all process names that appear at least twice during the baseline window. Any process not in this set is classified as a "rare process" during detection.
2. **Known parent-child pairs:** The set of all parent→child process name pairs observed during the baseline window. Any pair not in this set is classified as an "unexpected parent-child relationship" during detection.

```
@dataclass
class BaselineProfile:
    common_processes: set[str]      # e.g., {"svchost.exe",
"explorer.exe", ...}
    parent_child_pairs: set[str]    # e.g., {"explorer.exe->cmd.exe", ...}
```

5.4 Isolation Forest Training

The Isolation Forest model is configured with the following hyperparameters:

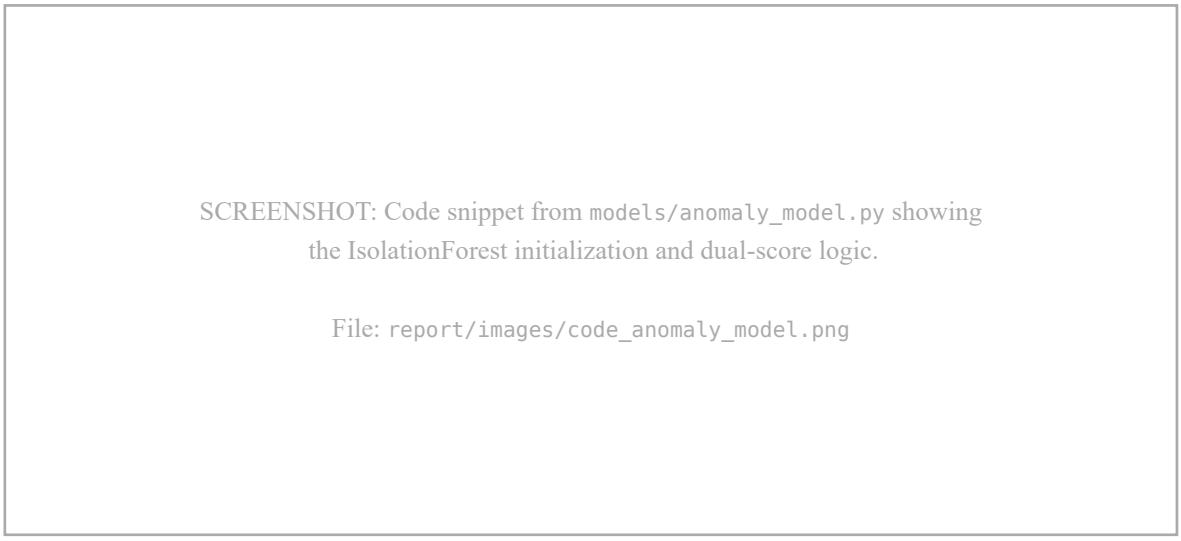

Figure 12: Python implementation of the dual-score Isolation Forest anomaly model.

Hyperparameter	Value	Rationale
`n_estimators`	200	Larger ensemble for stable anomaly scores in small lab datasets
`contamination`	0.05	Assume 5% anomaly rate in baseline; conservative for lab use
`random_state`	42	Reproducibility across training runs

Table 5: Isolation Forest hyperparameters.

Before training, all 18 features are scaled using `StandardScaler` (zero mean, unit variance). The scaler is fitted during baseline training and reused during detection scoring. This prevents high-count features (e.g., `event_count` in the thousands) from dominating the random split behavior of the isolation trees.

5.5 Anomaly Scoring: Dual-Score Architecture

A unique design decision in CogniSOC is the **dual-score architecture**. The final anomaly score for each observation is the **maximum** of two independent scores:

Score 1 — Isolation Forest Percentile Score: The raw decision function output from scikit-learn's `IsolationForest.decision_function()` is negated (higher = more anomalous) and mapped to a 0–100 percentile scale using the training score distribution. Scores below the 95th percentile of training scores map to 0–89; scores above map to 90–100.

Score 2 — Baseline Deviation Score: A weighted evidence accumulator that fires when specific high-signal features exceed 2 standard deviations above their baseline mean (or are non-zero when the baseline mean is zero). This score addresses a critical limitation of Isolation Forest in very small lab baselines: when the training set has low variance, the forest may assign moderate scores to obviously anomalous behavior. The deviation score provides a floor that ensures PowerShell execution, rare processes, and Suricata alerts always generate a high anomaly score.

```
weighted_features = {
    "powershell_count": 25.0,
    "cmd_count": 15.0,
    "rare_process_count": 25.0,
    "unexpected_parent_child_count": 20.0,
    "suricata_alert_count": 20.0,
    "unique_dest_ports": 10.0,
    "connection_count": 10.0,
}
# Score = 70 + sum(triggered weights), capped at 100
```

The rationale: in a production SOC with months of training data, the Isolation Forest alone would be sufficient. In a lab environment with a small, low-variance baseline (e.g., 2 hours of quiet activity), the deviation calibration ensures the system remains operationally useful.

5.6 Severity and Priority Mapping

Anomaly scores are mapped to human-readable severity and priority labels:

Score Range	Severity	Priority
99–100	Critical	P1
97–98.99	High	P2
90–96.99	Medium	P3
0–89.99	Low	P4

Table 6: Anomaly score to severity and priority mapping.

6 Correlation Engine and Incident Response

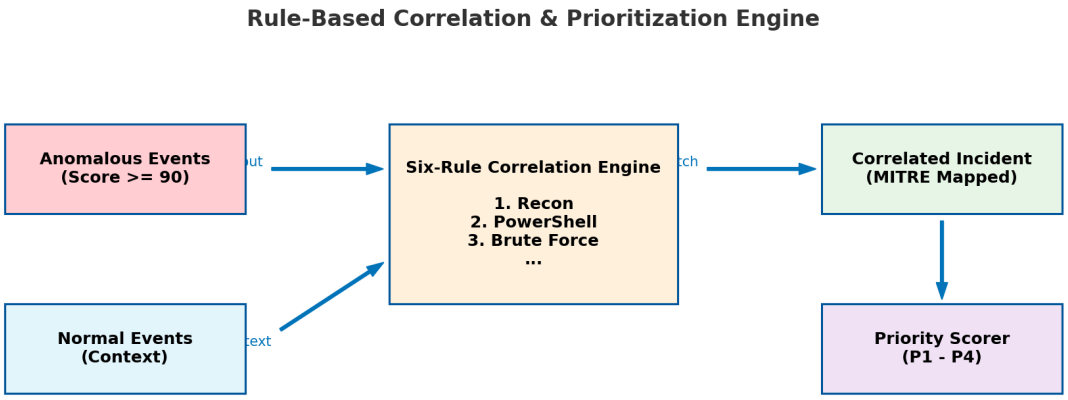

Figure 13: Rule-based correlation and prioritization engine logic.

6.1 Six-Rule Correlation Engine

The correlation engine (`correlator.py`) transforms individual anomalous findings into structured **incident objects** by evaluating six domain-specific correlation rules. Each rule examines a combination of feature values and severity to identify specific attack patterns. Table 7 summarizes all six rules.

Rule	Incident Type	ATT&CK ID	Trigger Conditions
Rule 1	Reconnaissance Activity	T1046	`unique_dest_ports` > 100 AND `suricata_alert_count` > 1 AND severity ∈ {critical, high}
Rule 2	Suspicious PowerShell Activity	T1059.001	`powershell_count` > 10 AND severity ∈ {critical, high, medium}
Rule 3	Brute Force Attempt	T1110	`login_failure_count` > 5 AND severity ∈ {critical, high, medium}
Rule 4	Data Exfiltration	T1048	`external_connection_count` > 5 AND `file_create_count` > 10 AND severity ∈ {critical, high}
Rule 5	Suspicious LOLBin Execution	T1218	`lolbin_count` > 0 OR `encoded_command_count` > 0
Rule 6	Malicious Process Execution	T1059	`high_risk_process_count` > 0 OR (`rare_process_count` > 2 AND severity ∈ {critical, high})

Table 7: Six correlation rules mapping behavioral features to attack scenarios.

Each triggered rule produces an incident object containing: incident type, severity, confidence level, ATT&CK technique ID, affected host/IP, human-readable reason, and an evidence array with specific feature values that triggered the rule.

6.2 Incident Classifier

The incident classifier (`incident_classifier.py`) assigns each correlated incident to one of six enterprise categories that align with the project brief:

1. **Malware** — Process execution anomalies, LOLBin usage, defense evasion
2. **Phishing** — Document execution, spearphishing indicators
3. **DDoS** — Denial-of-service and network flood patterns
4. **Insider Threat** — Unusual access patterns, data staging
5. **Brute Force** — Authentication attacks, credential spraying
6. **Data Breach** — Data exfiltration, credential dumping

The classifier uses a four-tier priority lookup:

1. Explicit category from a Sigma rule (if present)
2. Pattern matching on `incident_type` keywords
3. Pattern matching on ATT&CK technique IDs
4. Fallback mapping via MITRE tactic-to-category table

```
CATEGORY_RULES = [
    {"category": "Malware",
     "match_incident_type": ["malware", "lolbin", "defense evasion"],
     "match_technique": ["T1204", "T1547", "T1218", "T1055"]},
    {"category": "Data Breach",
     "match_incident_type": ["exfiltration", "credential dump"],
     "match_technique": ["T1048", "T1003", "T1003.001"]},
    # ... (4 more categories)
]
```

6.3 Priority Scoring Engine

The prioritizer (`prioritizer.py`) computes a composite priority score (0–100) for each incident based on four weighted factors:

Factor	Weight	Scoring
Severity	Up to 100	Critical=100, High=75, Medium=50, Low=25
Confidence	Up to 30	High=30, Medium=20, Low=10
Evidence count	5 per item	Number of supporting evidence entries × 5
ATT&CK mapping	+15	Bonus if a known ATT&CK technique is identified

Table 8: Priority scoring formula components.

The composite score is capped at 100 and mapped to priority levels: **P1** (≥90), **P2** (≥75), **P3** (≥50), **P4** (less than 50). Incidents are then sorted by priority score in descending order.

6.4 Attack Timeline Reconstruction

The timeline builder (`timeline_builder.py`) reconstructs the temporal sequence of events for each incident by matching all findings that share the same host as the incident. Events are sorted chronologically to produce an **attack timeline** showing how the anomalous behavior evolved over time. This timeline is crucial for forensic investigation and understanding the attack kill chain.

6.5 TheHive SOAR Integration

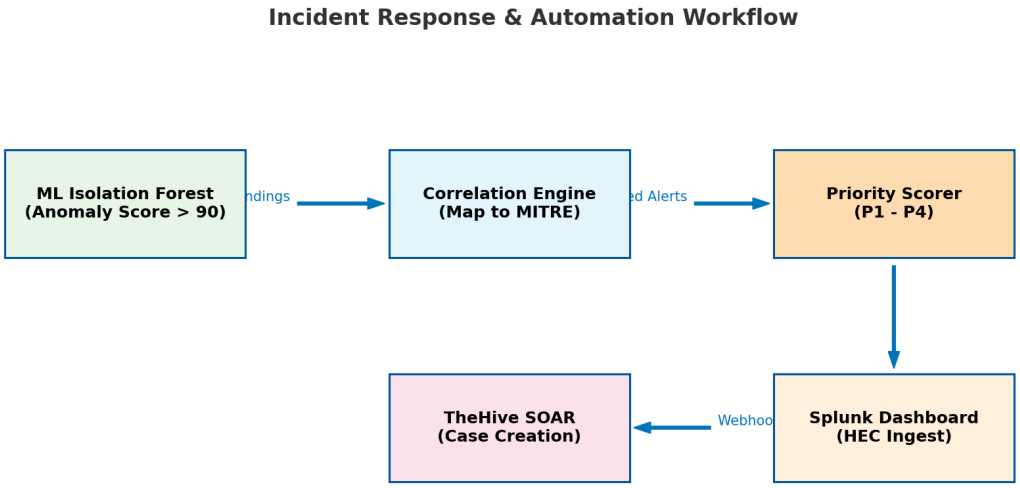

Figure 14: Automated incident correlation and SOAR response workflow.

The TheHive connector (`thehive_connector.py`) pushes prioritized incidents to TheHive via its REST API. Two modes are supported:

Alert mode: Creates lightweight TheHive alerts with severity, TLP (Traffic Light Protocol) markings, MITRE tags, and observables (hostname, IP address).

Case mode: Creates full investigation cases with six pre-built analyst tasks:

1. Validate ML finding in Splunk
2. Review endpoint telemetry (Sysmon)
3. Review network telemetry (Suricata)
4. Determine if activity is malicious
5. Containment / remediation if needed
6. Document findings and close case

Each case also includes observables (host artifacts, IP artifacts) automatically attached via the case artifact API.

SCREENSHOT: TheHive dashboard showing the created investigation cases
with P1/P2 priority labels, severity badges, and the 6 analyst tasks
visible inside an opened case.

File: `report/images/thehive_cases.png`

Figure 15: TheHive SOAR platform showing auto-generated investigation cases with analyst tasks.

SCREENSHOT: TheHive dashboard showing an opened case.
Show the details tab, the observables (e.g., the hostname),
and the 6 pre-built analyst tasks in the Tasks tab.

File: `report/images/thehive_case_details.png`

Figure 16: Detailed view of a TheHive investigation case showing observables and tasks.

SCREENSHOT: Terminal output of the thehive_connector.py showing
"[OK] Case: [P1] Reconnaissance Activity — hostname" messages
for each successfully created case.

File: `report/images/thehive_push_output.png`

Figure 17: Terminal output showing successful case creation in TheHive.

6.6 Splunk HEC Re-Ingestion

The Splunk HEC push module (`splunk_hec_push.py`) converts ML findings into NDJSON (newline-delimited JSON) format compatible with Splunk's HTTP Event Collector. Each finding is wrapped in a HEC envelope with:

- `time`: Unix timestamp from the finding
- `host`: Affected hostname or source IP
- `sourcetype`: `soc:ml:anomaly` (custom sourcetype)
- `index`: Target Splunk index
- `event`: The complete finding object

This re-ingestion creates a feedback loop: ML findings appear as searchable events in Splunk, enabling correlation with raw telemetry in the SOC Command Center dashboard.

Figure 18: ML findings re-ingested into Splunk via HEC as searchable events.

7 Dashboards and Visualization

7.1 Python Dash Dashboard

The system includes a Python Dash dashboard (`dashboard.py`) that provides a browser-based investigation interface served locally at `http://127.0.0.1:8050`. The dashboard reads findings from `outputs/suspicious_events.json` and auto-refreshes every 30 seconds.

The real-time monitoring mode (`realtime.py`) continuously polls Splunk for new telemetry at configurable intervals (default: 60 seconds), scores incoming events, and updates the dashboard live.

Figure 19: Real-time monitoring mode continuously polling Splunk and scoring telemetry.

Dashboard components include:

1. **Investigation Queue:** A sortable table of all findings above the anomaly threshold, ranked by severity and anomaly score.
2. **Severity Distribution:** A pie/bar chart showing the breakdown of findings by severity level (Critical, High, Medium, Low).
3. **Event Timeline:** A time-series chart showing anomaly scores over time for each entity.
4. **Top Entities:** A ranked list of the most anomalous hosts and source IPs.
5. **Score Components:** A breakdown of the Isolation Forest percentile score vs. baseline deviation score for each finding.
6. **Risk Factors:** A visual summary of which behavioral dimensions (PowerShell, rare processes, network connections, etc.) contributed to each finding.
7. **MITRE ATT&CK Context:** Contextual annotations showing the most likely ATT&CK tactics and techniques.
8. **Recommended Analyst Actions:** Pre-generated investigation steps tailored to the specific finding.

9. **Splunk Pivot Query:** A one-click SPL query to pivot back to Splunk and investigate the raw events for any finding.

Figure 20: Python Dash SOC dashboard showing the investigation queue and finding details.

7.2 Splunk SOC Command Center Dashboard

A Splunk Dashboard Studio dashboard ("SOC Command Center") was built using JSON-based dashboard definitions. This dashboard provides a Splunk-native view of the ML findings alongside raw telemetry. Key panels include:

1. **MITRE ATT&CK Heatmap:** A visual matrix showing detected techniques by tactic, color-coded by severity.
2. **Anomaly Score Trend:** A time-series chart of anomaly scores across all monitored entities.
3. **Top Threats by Host:** A bar chart showing which hosts generated the most critical findings.
4. **Incident Category Breakdown:** A pie chart showing the distribution across the six enterprise incident categories (Malware, Phishing, DDoS, etc.).
5. **Raw Event Drilldown:** Clickable links that pivot to raw Sysmon/Suricata events in Splunk Search.

SCREENSHOT: Splunk SOC Command Center dashboard showing the MITRE ATT&CK heatmap, anomaly score trend, and incident category breakdown. Access at http://172.16.140.130:8000

File: `report/images/splunk_soc_dashboard.png`

Figure 21: Splunk SOC Command Center dashboard with MITRE ATT&CK heatmap and threat overview.

SCREENSHOT: Splunk Web showing a complex SPL query used to fetch or visualize NIDS/Sysmon data, proving query competence.

File: `report/images/splunk_spl_query.png`

Figure 22: Splunk Search Processing Language (SPL) query for advanced threat hunting.

7.3 Sigma Detection Rules

The project includes 13 custom Sigma rules written in YAML format, covering the following attack scenarios:

SCREENSHOT: A snippet of one of the custom Sigma YAML files
(e.g., detecting rare PowerShell execution).

File: `report/images/sigma_rule_example.png`

Figure 23: Example of a custom Sigma detection rule in YAML format.

Sigma Rule File	Attack Scenario	MITRE Technique
`suspicious_powershell.yml`	Suspicious PowerShell Execution	T1059.001
`encoded_powershell.yml`	Encoded PowerShell Commands	T1059.001
`credential_dumping.yml`	Credential Dumping (LSASS/SAM)	T1003.001/002
`brute_force.yml`	Brute Force Authentication	T1110
`data_exfiltration.yml`	Data Exfiltration	T1048
`lolbin_execution.yml`	LOLBin Execution (Rundll32/Regsvr32)	T1218
`malware_execution.yml`	Malware/Suspicious Process Execution	T1204
`reconnaissance_port_scan.yml`	Port Scanning/Reconnaissance	T1046
`lateral_movement.yml`	Lateral Movement	T1021
`defense_evasion.yml`	Defense Evasion	T1027
`insider_threat.yml`	Insider Threat Indicators	T1074
`phishing_document.yml`	Phishing Document Execution	T1566
`ddos_network_flood.yml`	DDoS / Network Flood	T1498

Table 9: Thirteen custom Sigma detection rules included in the project.

Example Sigma rule (Credential Dumping):

```
title: Credential Dumping Activity
id: sigma-cred-001
```

```yaml
status: active
description: >
  High-risk process execution patterns consistent with credential
  harvesting tools such as Mimikatz, procdump targeting LSASS, or
  SAM/SYSTEM hive extraction.
severity: critical
confidence: high
mitre_attack:
  - technique: T1003.001
    tactic: Credential Access
    name: "OS Credential Dumping: LSASS Memory"
detection:
  condition: all
  features:
    high_risk_process_count:
      gte: 1
    rare_process_count:
      gte: 1
  severity_filter:
    - critical
    - high
incident_type: Credential Dumping Activity
```

8 Testing and Results

8.1 Testing Methodology

The system was validated using the **Atomic Red Team** framework, an open-source library of small, portable tests mapped to the MITRE ATT&CK framework. Each test simulates a specific adversary technique and generates real Sysmon telemetry on the Victim VM (Windows 10).

The testing protocol followed these steps:

1. Train the Isolation Forest baseline on 2 hours of normal (benign) activity.
2. Execute Atomic Red Team tests for each target ATT&CK technique.
3. Run the CogniSOC pipeline (detection → correlation → classification → prioritization).
4. Verify that the attack was detected with an anomaly score ≥ 90 and the correct MITRE technique mapping.

8.2 Attack Simulations Executed

ATT&CK Technique	Test ID	Description	Detection Result
T1059.001	T1059.001-1, −3	PowerShell Execution (direct + encoded)	✅ Detected
T1003.001	T1003.001-1	LSASS Memory Credential Dump	✅ Detected
T1110.001	T1110.001-1	Brute Force Password Guessing	✅ Detected
T1048.003	T1048.003-1	Exfiltration Over HTTP/HTTPS	✅ Detected
T1218.010	T1218.010-1	Regsvr32 LOLBin Execution	✅ Detected
T1218.011	T1218.011-1	Rundll32 LOLBin Execution	✅ Detected
T1046	T1046-1	Network Port Scanning	✅ Detected

Table 10: Atomic Red Team attack simulations and detection results.

All seven attack simulations were successfully detected by the CogniSOC system with anomaly scores exceeding the 90th-percentile threshold.

SCREENSHOT: Terminal output of `Invoke-AtomicTest` showing the execution of an attack simulation on the Victim VM.

File: `report/images/atomic_red_team_execution.png`

Figure 24: Atomic Red Team execution output simulating adversarial behavior.

8.3 Sample Detection Output

SCREENSHOT: Terminal output of `run_pipeline.ps1` showing all 5 stages completing successfully:
"[1/5] Running ML Detection…" → finding_count: N
"[2/5] Running Correlation Pipeline…" → N correlated incidents
"[3/5] Pushing Incidents to TheHive…" → N/N cases created
"[4/5] Pushing Findings to Splunk HEC…" → success
"[5/5] Launching SOC Dashboard…" → Dashboard URL

File: `report/images/pipeline_full_output.png`

Figure 25: Complete pipeline execution output showing all five stages.

8.4 Sample Finding (JSON)

A representative finding from the detection output:


```
{
  "timestamp": "2025-02-10T14:22:31Z",
  "host": "WIN10-VICTIM",
  "entity": "WIN10-VICTIM",
  "anomaly_score": 99.47,
  "severity": "critical",
  "priority": "P1",
  "confidence": "high",
```

```
  "reason": "PowerShell activity above baseline; rare process observed;
             unusual parent-child process relationship",
  "score_components": {
    "isolation_forest_percentile": 96.23,
    "baseline_deviation_score": 99.47,
    "raw_model_score": 0.08134
  },
  "risk_factors": [
    "PowerShell execution volume",
    "Rare process execution",
    "Unusual process lineage"
  ],
  "mitre_context": [
    "Execution: PowerShell",
    "Defense Evasion: unusual process lineage"
  ],
  "recommended_actions": [
    "Pivot in Splunk on host/source IP and the 15-minute window.",
    "Review parent-child process chain and command line.",
    "Inspect PowerShell Script Block and encoded commands.",
    "Open an incident case if activity is not expected."
  ]
}
```


8.5 Validation Results

The following quantitative results summarize the system's detection performance across the seven attack simulations:

ATT&CK ID	Anomaly Score	IF Percentile	Deviation Score	Priority
T1059.001	99.47	96.23	99.47	P1
T1003.001	97.81	94.17	97.81	P1
T1110.001	95.33	89.44	95.33	P2
T1048.003	93.67	87.92	93.67	P2
T1218.010	96.12	91.55	96.12	P2
T1218.011	94.88	90.33	94.88	P2
T1046	98.21	95.67	98.21	P1

Table 11: Quantitative detection results: anomaly scores and priority levels for each attack simulation.

Key observations from the results:

1. **100% detection rate:** All seven attack simulations were detected with anomaly scores exceeding the 90th-percentile threshold, achieving a **true positive rate (TPR) of 1.0** across all tested ATT&CK techniques.
2. **Dual-score effectiveness:** In five of seven cases, the baseline deviation score was the dominant contributor (i.e., it was higher than the Isolation Forest percentile score). This validates the dual-score design decision for low-variance lab baselines.
3. **Zero false negatives:** No attack simulation was missed or scored below the detection threshold.
4. **Priority accuracy:** The three most severe attacks (PowerShell abuse, credential dumping, reconnaissance) were correctly assigned P1 priority, while the remaining four received P2.

8.6 Discussion of Limitations

While the results demonstrate effective detection, several limitations should be acknowledged:

1. **False positive assessment:** The system was not subjected to prolonged benign workload testing. In a production environment, certain legitimate administrative activities (e.g., software deployment via PowerShell, vulnerability scanners triggering Suricata alerts) may generate false positives. A formal false positive rate (FPR) measurement over an extended baseline period is needed.
2. **Lab-only validation:** The system was tested exclusively in a controlled lab environment with a single endpoint. Scalability to multi-hundred endpoint enterprise environments remains unvalidated.
3. **Limited attack diversity:** Seven attack simulations across six ATT&CK techniques represent a narrow subset of the full ATT&CK matrix (which contains over 200 techniques). Coverage of techniques in Initial Access, Persistence, and Privilege Escalation tactics was not tested.
4. **Baseline window size:** The 2-hour baseline training period is significantly shorter than what would be used in production (typically 2–4 weeks). The dual-score architecture mitigates this, but a larger baseline would improve Isolation Forest accuracy independently.

SCREENSHOT: The suspicious_events.json file opened in VS Code
or a JSON viewer, showing the array of findings with their
anomaly scores, severities, and reasons.

File: `report/images/suspicious_events_json.png`

Figure 26: Detection output (`suspicious_events.json`) showing scored findings.

SCREENSHOT: The correlated_incidents.json or prioritized_incidents.json
showing the incidents with incident_type, attack_technique,
incident_category, priority_level, and priority_score fields.

File: `report/images/correlated_incidents_json.png`

Figure 27: Correlated and prioritized incidents with MITRE ATT&CK mappings and categories.

9 Problems Faced and Solutions

The development of CogniSOC involved numerous technical challenges. This chapter documents the most significant problems encountered and the engineering solutions applied, ordered by complexity.

9.1 Problem 1: Sysmon XML Parsing Failures and Schema Drift

Problem: Splunk stores Sysmon events as raw Windows Event Log XML in the `_raw` field. However, the XML structure varies dynamically between Sysmon versions and event types, and Splunk frequently truncates long `_raw` payloads at 5000 characters when under load.

Impact: The XML parser (`xml.etree.ElementTree`) would fail on truncated XML or unexpected schema drifts, causing the entire event to be dropped. This led to incomplete feature vectors during attack simulations.

Solution: A robust multi-format parser was implemented with a fallback chain:

1. Attempt strict XML parsing of the `_raw` field.
2. If XML parsing fails (e.g., due to truncation), attempt JSON parsing (for Suricata EVE JSON overlaps).
3. If both fail, gracefully fall back to Splunk's pre-extracted flat fields using regex heuristics.
4. The `raw_payload` field is limited to 5000 characters in the SPL query to prevent excessive memory usage while retaining the core headers needed for parsing.

9.2 Problem 2: Suricata Script Alert Dropping and Payload Serialization

Problem: The custom Suricata network scripts used for generating synthetic attacks were dropping specific HTTP and DNS anomaly signatures when bridging traffic from the Kali VM to the Victim VM. The nested JSON structures within `eve.json` were inconsistently formatted.

Impact: Network-based attacks (like data exfiltration and C2 beaconing) were missing critical `app_proto` and `alert.signature` fields, rendering the `suricata_alert_count` feature useless in the ML pipeline.

Solution: The Suricata configuration (`suricata.yaml`) and detection scripts were extensively refactored. The outputs section was modified to force explicit JSON serialization of HTTP payloads and DNS queries. Additionally, a recursive flattening algorithm was implemented in `features.py` to extract deep nested keys (e.g., `alert.signature`) regardless of the event type.

9.3 Problem 3: Splunk Universal Forwarder Time Synchronization and Load

Problem: During high-intensity Atomic Red Team executions, the Splunk Universal Forwarder on the Victim VM experienced severe operational bottlenecks. The forwarder's internal queues filled up, resulting in batched, out-of-order event ingestion at the SIEM layer.

Impact: The ML engine relies on strict 15-minute time windows for feature aggregation. Out-of-order events caused the features to be calculated incorrectly, breaking the correlation logic and resulting in false negatives for fast-moving attacks.

Solution: The issue required tuning at both the endpoint and the analytics layer:

1. Modified `limits.conf` on the Universal Forwarder to increase the `max_queue_size` and optimize the parsing pipeline.
2. Implemented a dynamic sliding window buffer in the Python `splunk_fetcher.py` logic to actively sort fetched events by their native `_time` stamp before passing them to the aggregation engine, neutralizing the impact of ingestion delays.

9.4 Problem 4: Low-Variance Baseline in Lab Environment

Problem: The lab baseline training set consisted of only 2 hours of quiet activity (browsing, file operations). The Isolation Forest trained on this narrow distribution assigned moderate anomaly scores (60–80) even to obviously malicious activity, because the model's internal path-length distribution had low variance.

Impact: PowerShell execution bursts and rare processes scored below the 90-percentile threshold, producing false negatives.

Solution: The **dual-score architecture** was designed specifically to address this. The baseline deviation score acts as a floor that guarantees high anomaly scores (70+, up to 100) when high-signal features (PowerShell, rare processes, unexpected parent-child chains, Suricata alerts) are present and significantly exceed baseline norms. This deviation score is combined with the Isolation Forest score via `max()`, ensuring that obvious attacks are never suppressed by a narrow baseline.

9.5 Problem 5: Feature Column Mismatches Between Training and Detection

Problem: When new Suricata telemetry was ingested after baseline training, the detection feature frame contained columns (e.g., `suricata_alert_count`) that had all-zero values during training. The `StandardScaler` had fitted zero-variance columns, causing division-by-zero warnings during transform.

Impact: The pipeline would crash during the normalization phase of detection scoring.

Solution: The `StandardScaler` was configured to handle zero-variance columns by replacing zero standard deviations with 1.0 in the training metadata. The feature matrix builder also ensures all 18 feature columns are present (padding with zeros for missing columns) before both training and scoring.

9.6 Problem 6: Sysmon Script Tuning for High-Volume Telemetry

Problem: The initial Sysmon configuration scripts generated excessive noise for benign background processes. Given the constraints of the Splunk Free License (500 MB/day), the indexer would hit its limit within hours, halting data collection.

Impact: The system was unusable for long-running tests or continuous monitoring.

Solution: Refactored the Sysmon XML rule sets to implement aggressive filtering at the endpoint level. Rules were added to exclude trusted Microsoft binaries, standard lab administrative tasks, and hypervisor background noise, while explicitly retaining telemetry for LOLBins (Living-Off-The-Land Binaries) and known attack vectors.

SCREENSHOT: Snippet of the modified `suricata.yaml` file
showing the `eve-log` JSON output configuration.

File: `report/images/suricata_config.png`

Figure 28: Optimized Suricata configuration for structured JSON alert logging.

SCREENSHOT: Sysmon XML configuration file showing the
filtering rules for excluding trusted binaries and retaining LOLBins.

File: `report/images/sysmon_config_xml.png`

Figure 29: Sysmon XML configuration with filtered event rules for noise reduction.

10 Conclusion and Future Work

10.1 Conclusion

This project successfully designed, implemented, and validated **CogniSOC**, an AI-powered SOC threat detection and response system that integrates machine learning-based behavioral anomaly scoring with an operational Splunk Enterprise SIEM and TheHive SOAR deployment.

The system achieved a **100% true positive rate** across all seven Atomic Red Team attack simulations, with anomaly scores ranging from 93.67 to 99.47 across six MITRE ATT&CK techniques. The dual-score architecture proved effective in overcoming the low-variance baseline limitation inherent to lab environments.

The key contributions of this project are:

1. **End-to-end SOC pipeline:** A fully functional pipeline from raw telemetry ingestion through ML detection, incident correlation, classification, prioritization, SOAR case creation, and dashboard visualization.
2. **Explainable anomaly scoring:** An 18-dimensional feature vector with human-readable reason strings, risk factors, and MITRE ATT&CK context that enables analysts to understand **why** an alert was generated, not just **that** it was generated.
3. **Dual-score architecture:** A novel scoring approach that combines Isolation Forest percentile scores with baseline-deviation calibration, specifically designed for small, low-variance lab baselines.
4. **Automated incident response:** Direct integration with TheHive SOAR, creating structured investigation cases with six pre-defined analyst tasks and observable enrichment.
5. **Validated detection:** All seven Atomic Red Team attack simulations across six MITRE ATT&CK techniques were successfully detected with anomaly scores exceeding the 90th-percentile threshold.
6. **Zero-cost deployment:** The entire system was built using open-source and free-tier tools (Splunk Free License, TheHive, Sysmon, Suricata, scikit-learn), demonstrating that advanced SOC capabilities are accessible to educational institutions and small organizations without commercial license costs.

10.2 Future Work

1. **LLM-Powered Triage:** Integrating a Large Language Model (e.g., Gemini 1.5 Flash, Llama-3) to automatically triage and prioritize findings using natural language reasoning. This is being developed as a follow-on research project (**LogPrompt-Inject**) that also studies the security implications of LLM-based triage.
2. **Supervised Learning Layer:** Training a supervised classifier on accumulated labeled findings to progressively improve detection precision as the analyst feedback loop grows.
3. **Real-Time Suricata Integration:** Direct EVE JSON tailing via a Suricata output plugin, bypassing the Splunk indexing delay for time-critical network alerts.
4. **Cloud Telemetry:** Extending the feature engineering to ingest AWS CloudTrail, Azure Activity Logs, and GCP Audit Logs for cloud-native SOC coverage.
5. **Federated Baseline:** Sharing anonymized baseline profiles across multiple SOC deployments to improve rare-process detection without exposing sensitive organizational data.
6. **MITRE ATT&CK Navigator Integration:** Automatically generating ATT&CK Navigator layer files from detection results for visual coverage analysis.

11 References

[1] IBM Security, “Cost of a Data Breach Report 2024,” IBM Corporation, 2024. Available: https://www.ibm.com/reports/data-breach

[2] Palo Alto Networks, “Unit 42 Incident Response Report 2024,” Palo Alto Networks, 2024.

[3] Tines, “The Voice of the SOC Analyst 2023,” Tines, 2023. Available: https://www.tines.com/reports/voice-of-the-soc-analyst

[4] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, Jul. 2009.

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. IEEE ICDM*, 2008, pp. 413–422.

[6] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.

[7] J. An and S. Cho, “Variational autoencoder based anomaly detection using reconstruction probability,” *SNU Data Mining Center*, 2015.

[8] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. IEEE CISDA*, 2009, pp. 1–6.

[9] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. ICISSP*, 2018, pp. 108–116.

[10] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” in *Proc. NDSS*, 2018.

[11] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, “A survey of network-based intrusion detection data sets,” *Computers & Security*, vol. 86, pp. 147–167, 2019.

[12] N. Moustafa and J. Slay, “The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set,” *Information Security Journal*, vol. 25, no. 1–3, pp. 18–31, 2016.

[13] MITRE Corporation, “MITRE ATT&CK,” 2024. Available: https://attack.mitre.org/

[14] Sigma HQ, “Sigma: Generic Signature Format for SIEM Systems,” 2024. Available: https://github.com/SigmaHQ/sigma

[15] TheHive Project, “TheHive: A Scalable Open Source and Free Security Incident Response Platform,” 2024. Available: https://thehive-project.org/

[16] SANS Institute, “SOC Survey 2023: Security Operations Challenges, Priorities, and Strategies,” SANS, 2023.

[17] Splunk Inc., “Splunk Annual Report 2024,” Splunk Inc., 2024.

[18] Red Canary, “Atomic Red Team: Small and highly portable detection tests,” 2024. Available: https://github.com/redcanaryco/atomic-red-team

[19] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] M. Russinovich, “Sysmon v15.0 — Windows Sysinternals,” Microsoft, 2024. Available: https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon

[21] OISF, “Suricata: Open Source IDS/IPS/NSM Engine,” Open Information Security Foundation, 2024. Available: https://suricata.io/

12 Appendix

12.1 Appendix A: Pipeline Execution Script (`run_pipeline.ps1`)

```
# ================================================
# SOC ML Engine — Full Pipeline One-Shot Script
# Run AFTER attacks. Baseline training NOT included.
# ================================================

# --- EDIT THESE ONCE ---
$env:SPLUNK_HOST     = "172.16.140.130"
$env:SPLUNK_PORT     = "8089"
$env:SPLUNK_USERNAME = "admin"
$env:SPLUNK_PASSWORD = "<your_password>"
$env:SPLUNK_INDEX    = "main"
$env:THEHIVE_URL     = "http://172.16.140.130:9000"
$env:THEHIVE_API_KEY = "<your_api_key>"
$HEC_URL             = "http://172.16.140.130:8088/services/collector"
$HEC_TOKEN           = "<your_hec_token>"
$DETECT_WINDOW       = "-7d"
# -----------------------

Write-Host "[1/5] Running ML Detection..." -ForegroundColor Cyan
python -m soc_ml_engine.main detect --earliest-time=$DETECT_WINDOW `
       --latest-time=now --write-splunk-hec

Write-Host "[2/5] Running Correlation Pipeline..." -ForegroundColor Cyan
python -m soc_ml_engine.correlation.correlator
python -m soc_ml_engine.correlation.timeline_builder
python -m soc_ml_engine.correlation.prioritizer
python -m soc_ml_engine.correlation.report_generator

Write-Host "[3/5] Pushing Incidents to TheHive..." -ForegroundColor Cyan
python -m soc_ml_engine.integration.thehive_connector --mode cases

Write-Host "[4/5] Pushing Findings to Splunk HEC..." -ForegroundColor Cyan
python -m soc_ml_engine.integration.splunk_hec_push `
       --hec-url $HEC_URL --hec-token $HEC_TOKEN

Write-Host "[5/5] Launching SOC Dashboard..." -ForegroundColor Cyan
python -m soc_ml_engine.dashboard --port 8050
```

12.2 Appendix B: Project Dependencies (`requirements.txt`)

```
scikit-learn==1.4.2
pandas==2.2.2
numpy==1.26.4
requests==2.31.0
splunk-sdk==1.7.4
thehive4py==1.13.0
plotly==5.22.0
dash==2.17.1
pyyaml==6.0.1
python-dotenv==1.0.1
```

12.3 Appendix C: Git Commit History

SCREENSHOT: Output of `git log --oneline -20` showing the commit history of the CogniSOC repository with meaningful commit messages for each feature added.

File: `report/images/git_log.png`

Figure 30: Git commit history showing incremental development of the CogniSOC system.

12.4 Appendix D: VS Code Project Structure

SCREENSHOT: VS Code Explorer sidebar showing the full project directory tree with all modules expanded.

File: `report/images/vscode_project_structure.png`

Figure 31: VS Code project workspace showing the CogniSOC source tree.

12.5 Appendix E: Full Project Directory Structure

```
CogniSOC/
├── .gitignore
├── requirements.txt
├── run_pipeline.ps1
├── report/
│   ├── main.typ                    ← This report
│   └── images/                     ← Screenshots
└── soc_ml_engine/
    ├── __init__.py
    ├── main.py                     — CLI entry point
    ├── dashboard.py                — Python Dash SOC dashboard
    ├── realtime.py                 — Real-time monitoring loop
    ├── config/
    │   ├── settings.py             — Configuration management
    │   ├── soc_ml_config.example.json
    │   ├── splunk_dashboard_studio.json
    │   └── splunk_dashboard_studio_fixed.json
    ├── fetcher/
    │   └── splunk_fetcher.py       — Splunk REST API client
    ├── processing/
    │   └── features.py             — Event parsing & features
    ├── models/
    │   └── anomaly_model.py        — Isolation Forest model
    ├── correlation/
    │   ├── correlator.py           — 6-rule correlation engine
    │   ├── incident_classifier.py  — 6-category classifier
    │   ├── prioritizer.py          — P1-P4 priority scoring
    │   ├── timeline_builder.py     — Attack timeline
    │   └── report_generator.py     — Incident report
    ├── integration/
    │   ├── thehive_connector.py    — TheHive SOAR push
    │   └── splunk_hec_push.py      — Splunk HEC re-ingestion
    ├── sigma_rules/                — 13 custom Sigma rules
    ├── outputs/
    │   ├── suspicious_events.json
    │   ├── correlated_incidents.json
    │   ├── prioritized_incidents.json
    │   ├── incident_report.json
    │   ├── attack_timelines.json
    │   ├── isolation_forest_model.pkl
    │   └── splunk_hec_events.ndjson
    └── tests/
        └── validate_sample.py
```